

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №3

з дисципліни «Розробка мобільних
застосувань під Android»

Тема: «ДОСЛІДЖЕННЯ СПОСОБІВ ЗБЕРЕЖЕННЯ ДАНИХ»

Виконала:

студентка групи ІА-31

Кизим Єлизавета.

Київ 2026

Тема: Дослідження способів збереження даних.

Мета роботи: дослідити способи збереження даних (база даних, файлова система, тощо) та отримати практичні навички щодо використання сховищ даних.

Завдання: Написати програму під платформу Андроїд, яка доповнює програму, що розроблена за лабораторною роботою 2, роботою зі сховищами. Тобто при натисканні на кнопку «ОК» додатково:

- здійснюється запис результату взаємодії з інтерфейсом до сховища (файл або базу даних);
- користувач інформується відповідним повідомленням щодо успішності запису.

Також інтерфейс необхідно доповнити кнопкою «Відкрити», натискання на яку призводить до переходу на іншу Діяльність, у якій відображається вміст даних, що зберігаються у сховищі. Якщо дані відсутні (сховище пусте) відобразити відповідне повідомлення. За бажанням можна додатково реалізувати оновлення та видалення даних зі сховища

Варіант 12:

12.	Вікно містить групу прапорів (площа і периметр), тобто чек-бокси, групу опцій (різні фігури), тобто радіо-батони, та кнопку «ОК». Вивести інформацію щодо вибору при натисканні на кнопку «ОК» у деяке текстове поле.
-----	---

Хід роботи

Фрагмент введення даних (InputFragment.kt): Виконує відображення форми вибору фігури, валідацію вводу та ініціює запис результатів у внутрішнє файлове сховище пристрою.

Як працює: При натисканні на кнопку «ОК» скрипт формує текстовий рядок результату і не лише передає його у SharedViewModel, а й викликає метод saveDataToFile(). Цей метод відкриває файл history.txt у режимі дозапису

(Context.MODE_APPEND) та додає новий запис разом із візуальним роздільником. Про успішність запису користувач інформується через спливаюче повідомлення Toast. При натисканні на нову кнопку «Історія» створюється Intent для переходу на екран історії. Для оптимізації роботи з пам'яттю ім'я файлу винесено у статичну константу через companion object.

Код InputFragment.kt:

```
package com.example.lab1

import android.content.Context
import android.content.Intent
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.*
import androidx.fragment.app.Fragment
import androidx.fragment.app.activityViewModels

class InputFragment: Fragment() {
    private val sharedViewModel: SharedViewModel by activityViewModels()

    companion object {
        private const val FILE_NAME = "history.txt"
    }

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        val view = inflater.inflate(R.layout.fragment_input, container, false)

        val radioGroupShapes =
view.findViewById<RadioGroup>(R.id.radioGroupShapes)
        val checkArea = view.findViewById<CheckBox>(R.id.checkArea)
```

```

val checkPerimeter = view.findViewById<CheckBox>(R.id.checkPerimeter)
val buttonOk = view.findViewById<Button>(R.id.buttonOk)

val buttonOpenStorage = view.findViewById<Button>(R.id.buttonOpenStorage)

buttonOk.setOnClickListener {
    val selectedShapeId = radioGroupShapes.checkedRadioButtonId
    val isAreaChecked = checkArea.isChecked
    val isPerimeterChecked = checkPerimeter.isChecked

    if (selectedShapeId == -1 || (!isAreaChecked && !isPerimeterChecked))
{
        Toast.makeText(requireContext(), "Будь ласка, завершіть введення
всіх даних!", Toast.LENGTH_LONG).show()
    } else {
        val shapeRadioButton =
view.findViewById<RadioButton>(selectedShapeId)
        val shapeName = shapeRadioButton.text.toString()
        val selectedParams = mutableListOf<String>()

        if (isAreaChecked) selectedParams.add("Площа")
        if (isPerimeterChecked) selectedParams.add("Периметр")

        val paramsText = selectedParams.joinToString(" та ")

        val resultString = "Обрана фігура: $shapeName\nОбрані параметри:
$paramsText"

        sharedViewModel.resultData = resultString

        saveDataToFile(resultString)
        Toast.makeText(requireContext(), "Дані успішно збережено!",
Toast.LENGTH_SHORT).show()

        radioGroupShapes.clearCheck()
        checkArea.isChecked = false
        checkPerimeter.isChecked = false

        parentFragmentManager.beginTransaction()

```

```

        .replace(R.id.fragment_container, ResultFragment())
        .addToBackStack(null)
        .commit()
    }
}

buttonOpenStorage.setOnClickListener {
    val intent = Intent(requireContext(), StorageActivity::class.java)
    startActivity(intent)
}

return view
}

private fun saveDataToFile(data: String) {
    try {
        val dataToWrite = "$data\n" + "-".repeat(18) + "\n"

        requireContext().openFileOutput(FILE_NAME, Context.MODE_APPEND).use {
            it.write(dataToWrite.toByteArray())
        }
    } catch (e: Exception) {
        e.printStackTrace()
        Toast.makeText(requireContext(), "Помилка при збереженні",
Toast.LENGTH_SHORT).show()
    }
}
}
}

```

Діяльність сховища (StorageActivity.kt): Слугує для відображення накопичених даних з файлу та надання можливості очищення сховища.

Як працює: Під час створення візуального інтерфейсу (у методі onCreate) діяльність звертається до внутрішньої пам'яті застосунку та намагається зчитати файл history.txt за допомогою потоку вводу openFileInput. Якщо файл існує та містить дані, вони виводяться у компонент TextView. Якщо файл порожній або відсутній, виводиться текст «Історія відсутня». Обробник кнопки «Очистити»

викликає метод `delete()` для файлу, а кнопка «Назад» викликає системний метод `finish()`, який програмно закриває поточну діяльність і повертає користувача на попередній екран.

Код `StorageActivity.kt`:

```
package com.example.lab1

import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import java.io.File

class StorageActivity : AppCompatActivity() {

    companion object {
        private const val FILE_NAME = "history.txt"
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_storage)

        val textViewContent = findViewById<TextView>(R.id.textViewStorageContent)
        val buttonClear = findViewById<Button>(R.id.buttonClearStorage)

        val buttonBack = findViewById<Button>(R.id.buttonBack)

        loadData(textViewContent)

        buttonClear.setOnClickListener {
            clearStorage(textViewContent)
        }
    }
}
```

```

        buttonBack.setOnClickListener {
            finish()
        }
    }

    private fun loadData(textView: TextView) {
        val file = File(filesDir, FILE_NAME)

        if (file.exists() && file.length() > 0) {
            try {
                val content = openFileInput(FILE_NAME).bufferedReader().use {
it.readText() }
                textView.text = content
            } catch (e: Exception) {
                e.printStackTrace()
                textView.text = "Помилка читання"
            }
        } else {
            textView.text = "Історія відсутня"
        }
    }

    private fun clearStorage(textView: TextView) {
        val file = File(filesDir, FILE_NAME)
        if (file.exists()) {
            file.delete()
            textView.text = "Історія відсутня"
            Toast.makeText(this, "Історія успішно очищена!",
Toast.LENGTH_SHORT).show()
        } else {
            Toast.makeText(this, "Тут і так пусто!", Toast.LENGTH_SHORT).show()
        }
    }
}

```

Макет екрана сховища (activity_storage.xml):

Виконує візуальне представлення екрана для виводу збереженої історії з файлу. Як працює: Побудований на основі LinearLayout. Для коректного відображення великих обсягів тексту (довгої історії виборів) TextView обгорнуто у компонент ScrollView, що забезпечує вертикальне прокручування. Також макет містить стилізовані кнопки для очищення історії та повернення на головний екран.

Код activity_storage.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp"
    android:background="@color/background_soft">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Історія:"
        android:textSize="22sp"
        android:textStyle="bold"
        android:textColor="@color/black"
        android:layout_marginBottom="16dp"/>

    <ScrollView
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:background="#FFFFFF"
        android:padding="12dp">

        <TextView
            android:id="@+id/textViewStorageContent"
```



```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="18sp"
        android:textColor="@color/black" />
</ScrollView>

<Button
    android:id="@+id/buttonClearStorage"
    android:layout_width="match_parent"
    android:layout_height="60dp"
    android:layout_marginTop="16dp"
    android:text="Очистити"
    android:backgroundTint="#F44336"
    android:textColor="#FFFFFF"
    android:textStyle="bold"
    android:textSize="18sp"/>

<Button
    android:id="@+id/buttonBack"
    android:layout_width="match_parent"
    android:layout_height="60dp"
    android:layout_marginTop="12dp"
    android:text="Назад"
    android:backgroundTint="@color/primary_blue"
    android:textColor="#FFFFFF"
    android:textStyle="bold"
    android:textSize="18sp"/>
</LinearLayout>

```

Головний макет форми введення (fragment_input.xml): Було модифіковано для підтримки нового функціоналу.

Як працює: До існуючої форми вибору фігур та параметрів було додано нову кнопку buttonOpenStorage («Історія»), яка розміщується безпосередньо під кнопкою підтвердження. Кольорова гама налаштована відповідно до загального стилю застосунку.

Код fragment_input.xml: (не весь файл, лише новий код)

```
<Button
    android:id="@+id/buttonOpenStorage"
    android:layout_width="match_parent"
    android:layout_height="60dp"
    android:layout_marginTop="12dp"
    android:textSize="18sp"
    android:textStyle="bold"
    android:text="Історія"
    android:backgroundTint="@color/primary_blue"
    android:textColor="@color/white" />
```

Результати роботи програми:

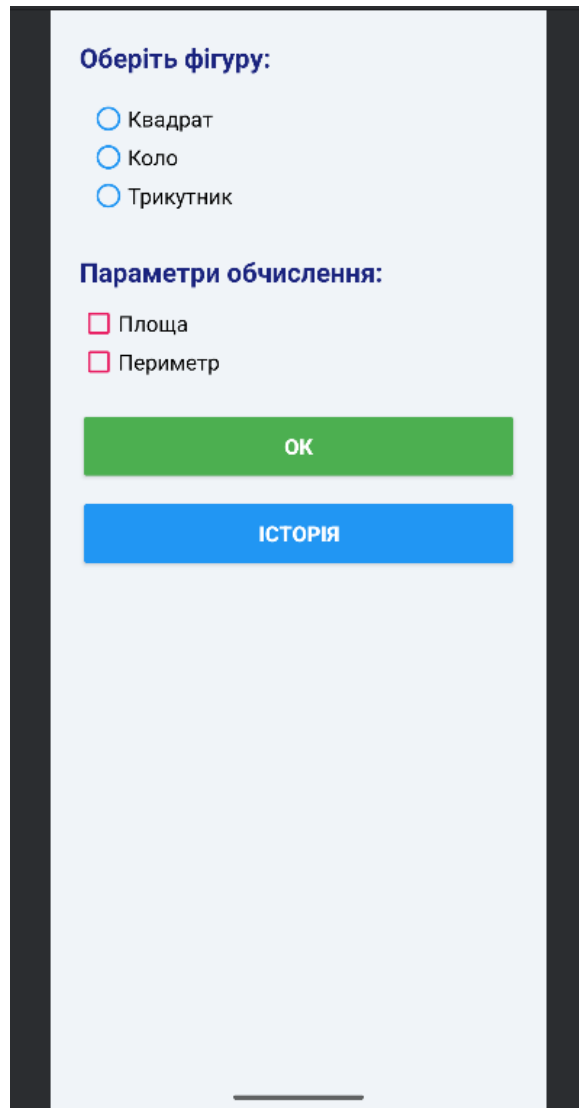


Рисунок 1: Головний екран з новою кнопкою "Історія"

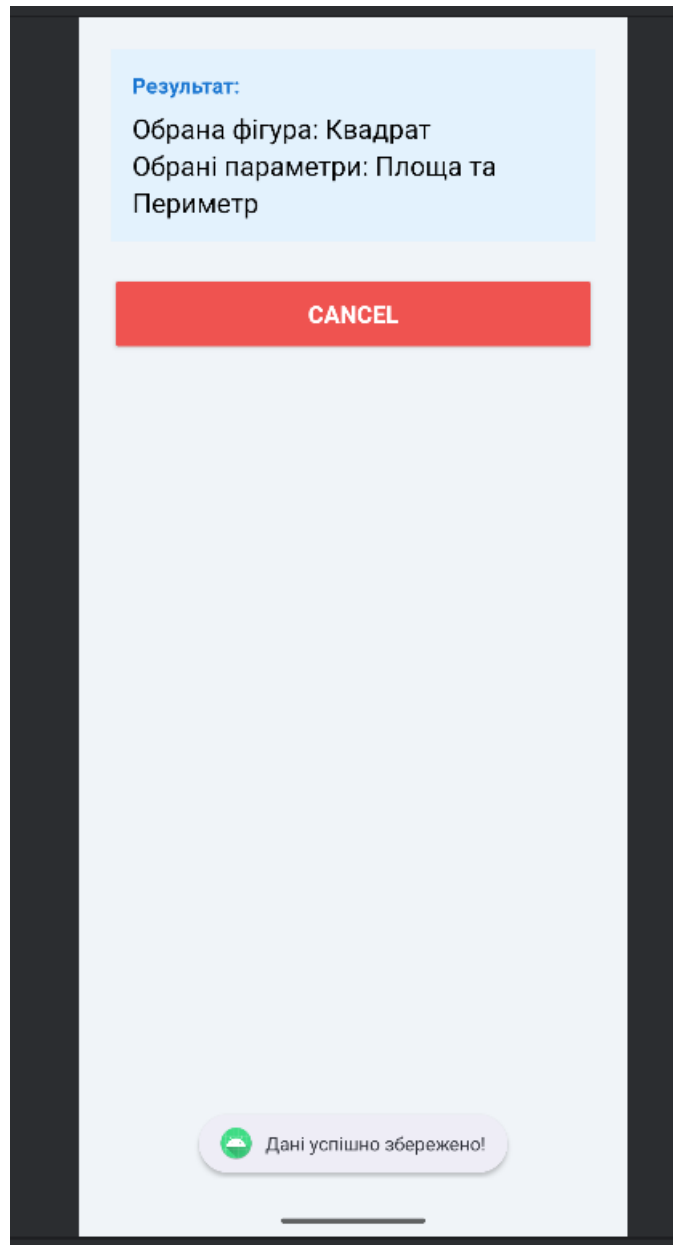


Рисунок 2: Повідомлення Toast "Дані успішно збережено!" після натискання "ОК"

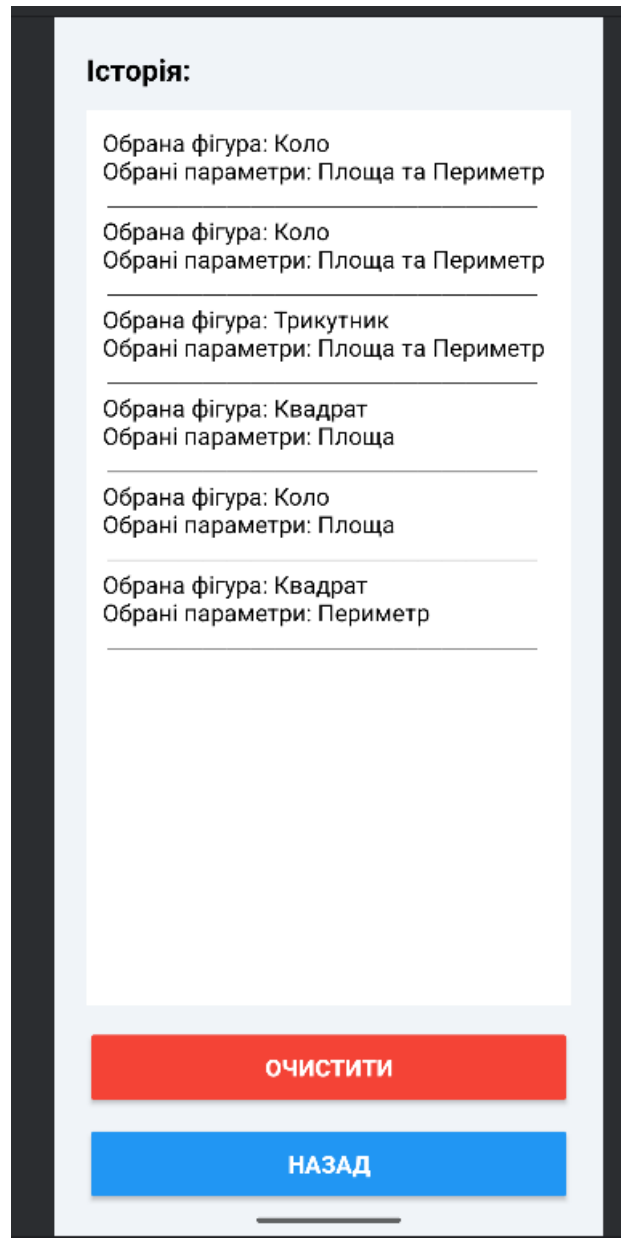


Рисунок 3: Екран StorageActivity з відображенням історії

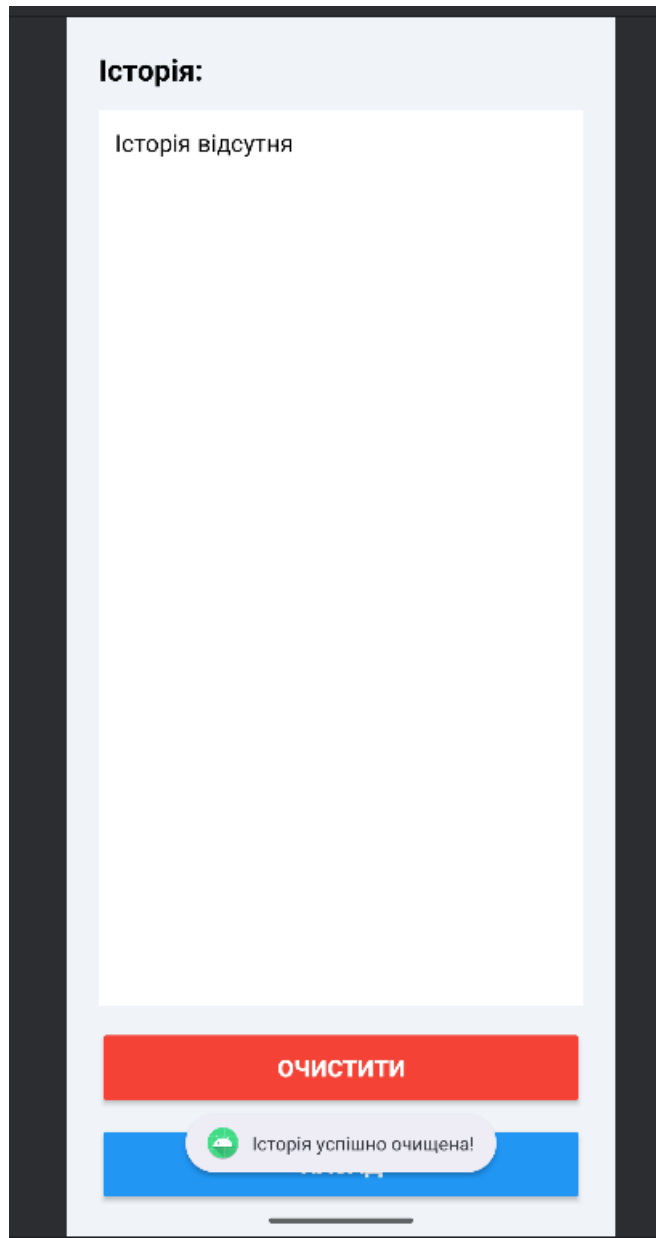


Рисунок 4: Екран StorageActivity після натискання кнопки "Очистити"

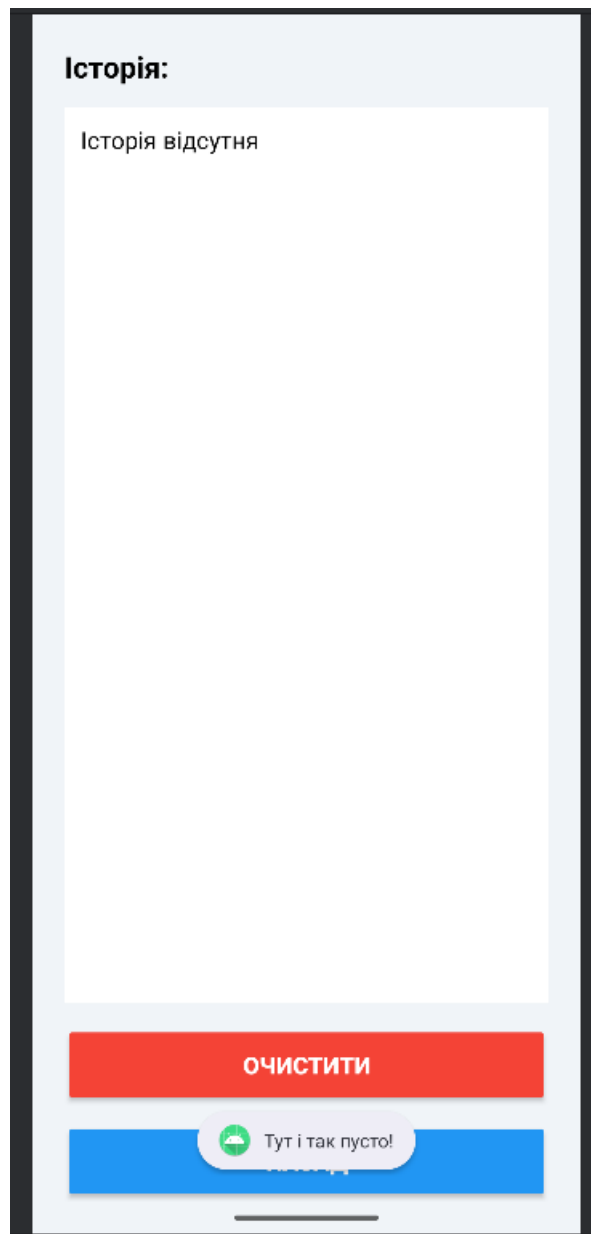


Рисунок 4: Екран StorageActivity після натискання кнопки "Очистити" при пустому сховищі

Висновок: Під час виконання лабораторної роботи було досліджено способи збереження даних в ОС Android на прикладі роботи з внутрішньою файловою системою (Internal Storage). Було успішно перенесено логіку застосунку з попередньої роботи та розширено її функціоналом безперервного запису текстових даних у файл та їх зчитування.

На практиці було реалізовано алгоритм збереження результатів взаємодії з інтерфейсом у текстовий файл за допомогою стандартних потоків вводу-виводу та

режиму `Context.MODE_APPEND`, що дозволило створити функціонал накопичення історії виборів.

КОНТРОЛЬНІ ПИТАННЯ

1. Опишіть як організована робота з налаштуваннями (даними у вигляді пари «ключ-значення»).

Робота з налаштуваннями в Android традиційно організовується за допомогою інтерфейсу `SharedPreferences` або новішого рішення `DataStore`. Дані зберігаються у XML-файлах або спеціальному бінарному форматі у приватній директорії застосунку. Для запису даних використовується об'єкт `Editor`, через який додаються пари ключ-значення, після чого зміни зберігаються асинхронно методом `apply` або синхронно методом `commit`. Читання відбувається безпосередньо через методи об'єкта `SharedPreferences` за ключем із можливістю вказати значення за замовчуванням на випадок відсутності збережених даних.

2. Опишіть типи сховищ файлів та причини їх використання.

В Android існують два основні типи сховищ файлів: внутрішнє та зовнішнє. Внутрішнє сховище використовується для збереження приватних даних застосунку, до яких інші програми та користувач не мають прямого доступу, і ці файли автоматично видаляються при видаленні застосунку. Зовнішнє сховище використовується для публічних файлів, таких як фотографії або завантаження, які можуть бути доступні іншим програмам та залишаються на пристрої навіть після видалення застосунку.

3. Опишіть процес роботи з файлами та файловою системою.

Процес роботи з файлами базується на стандартних класах мови Java для вводу-виводу, таких як `File`, `InputStream` та `OutputStream`. Для доступу до файлів застосунок отримує відповідні директорії через контекст, наприклад,

getFilesDir для внутрішнього сховища. У сучасних версіях Android робота зі спільними файлами здійснюється через Storage Access Framework та MediaStore, що вимагає використання ідентифікаторів URI замість прямих шляхів до файлів задля підвищення безпеки та ізоляції даних.

4. Опишіть процес роботи з базами даних SQLite за допомогою класу SQLiteOpenHelper, наведіть переваги та недоліки.

Робота з SQLite через клас SQLiteOpenHelper передбачає створення класу-спадкоємця з перевизначенням методів onCreate для початкового створення таблиць та onUpgrade для управління міграціями. Для виконання операцій використовуються об'єкти SQLiteDatabase, які дозволяють виконувати сирі SQL-запити або використовувати методи-обгортки. Перевагами цього підходу є відсутність сторонніх залежностей та повний контроль над запитами, проте до недоліків відносяться велика кількість шаблонного коду та відсутність перевірки SQL-запитів на етапі компіляції.

5. Як відбувається робота з БД за допомогою бібліотеки Room, наведіть переваги та недоліки.

Бібліотека Room є офіційною обгорткою над SQLite, яка забезпечує абстрактний рівень для роботи з базою даних через анотації. Процес включає створення сутностей, що представляють таблиці, об'єктів доступу до даних із запитами, та самого класу бази даних. Перевагами Room є перевірка SQL-запитів під час компіляції, значне зменшення шаблонного коду, спрощений механізм міграцій та вбудована підтримка реактивного програмування. Недоліком є лише необхідність вивчення додаткового API та незначні витрати часу на кодогенерацію під час збірки проєкту.

6. Наведіть характеристики екранів мобільних пристроїв.

Основними характеристиками екранів є фізична діагональ у дюймах, роздільна здатність, щільність пікселів, співвідношення сторін та частота

оновлення дисплея. Для розробки мобільних інтерфейсів критично важливою характеристикою також є логічна щільність, яка вимірюється в незалежних від щільності пікселях і дозволяє інтерфейсу виглядати однаково пропорційно на пристроях з різними екранами.

7. Наведіть класифікацію та відмінності технологій (типів) екранів мобільних пристроїв.

Технології мобільних екранів класифікуються на рідкокристалічні дисплеї та дисплеї на органічних світлодіодах. Рідкокристалічні екрани потребують загального підсвічування матриці, що забезпечує природні кольори, але має нижчий рівень контрастності. Екрани на органічних світлодіодах складаються з пікселів, кожен з яких є незалежним джерелом світла, що дозволяє досягти ідеального чорного кольору, високої контрастності та кращої енергоефективності при використанні темних тем інтерфейсу.

8. Наведіть поняття та характеристику сенсорних екранів.

Сенсорний екран — це пристрій введення та виведення інформації, що реагує на дотики користувача. Його головна характеристика полягає в усуненні необхідності використання зовнішніх маніпуляторів, дозволяючи користувачеві взаємодіяти з елементами інтерфейсу безпосередньо через жести. Це робить керування інтуїтивним, але вимагає адаптації розмірів елементів під пальці та врахування відсутності курсора.

9. Наведіть загальну класифікацію сенсорних екранів.

За принципом роботи сенсорні екрани класифікують на резистивні, ємнісні, інфрачервоні та поверхнево-акустичні. У смартфонах абсолютно домінують ємнісні екрани, які реагують на зміну електричної ємності при торканні струмопровідним об'єктом, підтримують розпізнавання кількох дотиків одночасно та мають високу прозорість. Резистивні екрани, що реагують на

фізичний тиск, зараз майже не використовуються в мобільних пристроях через низьку зносостійкість.

10. Наведіть рекомендації щодо розробки інтерфейсів для сенсорних екранів.

При розробці інтерфейсів рекомендується робити інтерактивні елементи достатньо великими та залишати між ними відступи для уникнення хибних натискань. Важливо забезпечувати миттєвий візуальний або тактильний зворотний зв'язок на кожен дотик, розташовувати ключові елементи керування в зоні досяжності великого пальця та уникати використання жестів, які можуть конфліктувати з системною навігацією пристрою.